# Лекция 4
# Механизмы оптимизации в ФП

GC

01

# Цена неизменяемости

Что происходит в памяти, когда вместо изменения данных создаются новые значения?

# Пример: loop

```
(loop [i 0, xs '()]
  (println
    "i =" i
    "xs =" xs
    "identity =" (System/identityHashCode xs))
  (when (< i 5)
    (recur (inc i)
           (conj xs i))))
```

Что напечатает программа?

## Что происходит с памятью?

Каждый `conj` возвращает новое значение

**Сколько объектов создано?**

# Новое значение вместо изменения

```
;; i = 0 xs = () identity = 686139129
;; i = 1 xs = (0) identity = 171972835
;; i = 2 xs = (1 0) identity = 1654488127
;; i = 3 xs = (2 1 0) identity = 1491187939
;; i = 4 xs = (3 2 1 0) identity = 1472966476
;; i = 5 xs = (4 3 2 1 0) identity = 415550945
```

**На каждой итерации новый список?**

# Пример: lazy-seq

```
(def xs
  (map (fn [x] (byte-array 10000))
       (range)))

(dorun (take 100000 xs)) ;; выделяется ~ 1ГБ памяти
```

## Что происходит с памятью?

```
(def xs ...)
```

**Можем ли мы быть уверены что xs больше не понадобится?**

# Пример: изменение состояния объекта

```
(loop [i 0
       p {:x 0 :y 0}]
  (if (< i 1000000)
    (recur (inc i)
           (-> p
               (update :x inc)          ;; создает новый!
               (update :y #(+ % 2))))) ;; создает еще!
    p))
```

## Что происходит с памятью?

```
(loop [...]
  (if (< i 1000000)
    (...
       (-> p
           (update :x ...)
           (update :y ...)
    ...)
```

## Сколько объектов создано после 1 млн итераций?

# Почему ФП не «тонет» в аллокациях?

```
immutable values
lazy sequences
new values instead of mutation
```

02

# Персистентные структуры данных

Как сохранять immutability без полного копирования коллекций?

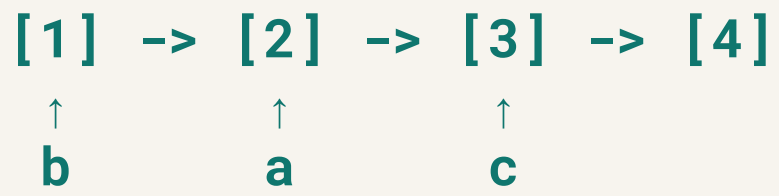
# Создание нового списка

```
a = (2 3 4 5)
b = (cons 1 a)
```

Копировать список целиком?

# Structural sharing

```
a = (2 3 4 5)
b = (cons 1 a)
c = (rest a)
```




Новая версия переиспользует старую структуру.

# Почему structural sharing безопасен?

```
a = (2 3 4 5)
b = (cons 1 a)
```

Почему a и b могут безопасно
использовать одни и те же узлы в Clojure?

# Persistent Data Structures

Persistent — постоянный, неизменный

**PDS — структура данных, которая при изменении сохраняет своё предыдущее состояние.**

## Принцип организации

1. Для двух разных состояний общая часть хранится в памяти в единственном экземпляре. Для каждого состояния отдельно хранится уникальная часть.

2. Принцип обобщается на произвольное число состояний.

# Два подхода к реализации PDS

**Fat-node**

**Один узел хранит историю значений
своих полей для нескольких версий.**

**Path copying**

**При изменении копируются узлы
на пути к изменяемому элементу.**

Fat-node: история внутри узлов.
Path copying: разные версии разделяют неизмененные узлы.

# Fat-node: как хранится изменение

```
v0 = [10 20 30 40]
v1 = (assoc v0 2 99)

Общие ячейки: [10]  [20]  [ ? ]  [40]
                           |
                  fat-node: индекс 2
                ┌──────────────────┐
                │   v0  =>  30     │
                │   v1  =>  99     │
                └──────────────────┘
```

Изменение записано в историю узла, а не в отдельную копию вектора.

# Fat-node: чтение с учётом версии

Один индекс, две операции чтения

```
get(v0, 2)  →  30

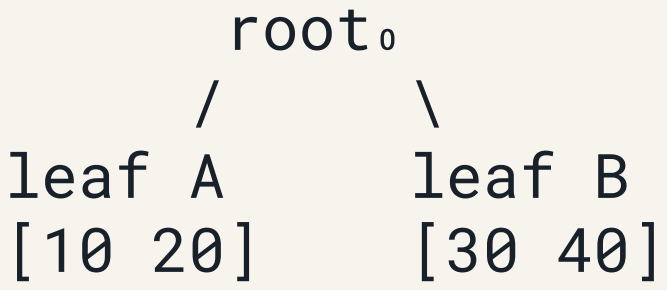
get(v1, 2)  →  99
```

**Один fat-node, но разные записи истории.**

Версия определяет, какое значение поля будет возвращено.
Сама структура остается общей (схематично).

# Подход Clojure: вектор в виде дерева

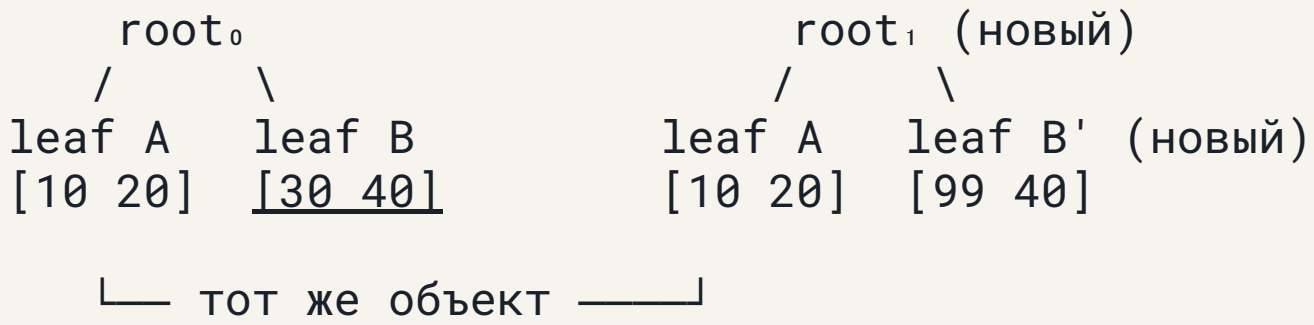
**$v_0$ = [10 20 30 40]**




Листья содержат блоки элементов, а внутренние узлы — ссылки на блоки.
В Clojure обычно до 32 элементов в листе; здесь показано по 2.

# Path copying: копируется путь

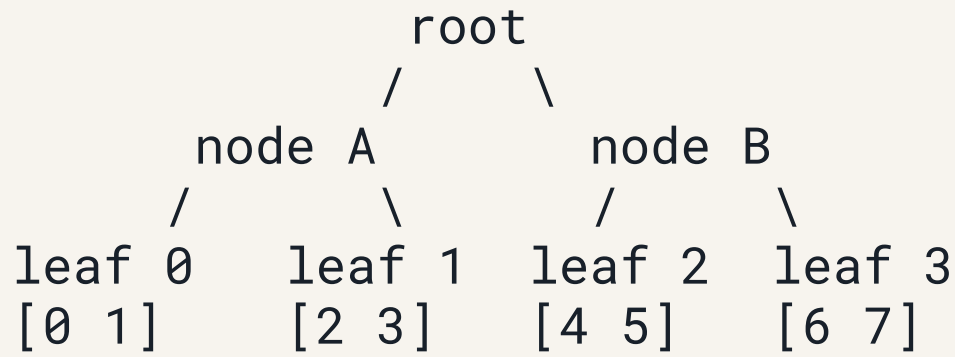
```
v1 = (assoc v0 2 99)
```




Копируются root и изменённый лист [30 40] → [99 40].
Лист [10 20] — один и тот же объект для обеих версий.

# Вектор из 8 элементов: блоки по 2

```
v = [0 1 2 3 4 5 6 7]
```




Каждый внутренний узел содержит до 2 ссылок; каждый лист — до 2 элементов.
При росте вектора появляется дополнительный уровень дерева.

# Устройство вектора Clojure

Дерево с ветвлением 32

Внутренние узлы — до 32 ссылок

Листья — до 32 элементов

Индекс разбивается на группы по 5 бит

Последние элементы хранятся в отдельном массиве `tail`

На практике дерево очень неглубокое

# Поиск элемента с индексом 42

$42_{10} = 00001\ 01010_2$

$00001_2 = 1$ ↓ корень: ссылка №1

$01010_2 = 10$ ↓ лист: элемент №10

$(42 >>> 5) \& 31 = 1$

$42 \& 31 = 10$

Для вектора с деревом из двух уровней (например, 128 элементов):
сначала выбираем лист №1, затем позицию №10 в нём.
Если индекс попадает в tail, поиск по дереву не нужен.

# Операции с вектором Clojure

**nth** – переход по дереву по группам битов
**assoc** – копирование пути до листа
**conj** – добавление через tail

# Почему 32?

$$32^4 = 1\ 048\ 576$$

Миллион элементов помещается в дерево очень небольшой глубины — поэтому $\log_{32} n$ практически выглядит как константа.

# Стоимость операций

```
list: first / cons   ↔  O(1)
list: nth            ↔  O(n)
vector: nth / assoc  ↔  O(log₃₂ n)
vector: conj         ↔  обычно O(1)
                     ↔  O(log₃₂ n), если tail заполнен
```

# Если важна производительность

```
(-> (transient []) ;; создаем мутабельный вектор
    (conj! 1)
    (conj! 2)
    persistent!)   ;; превращаем в персистентный
```

Локальная контролируемая мутабельность внутри реализации может сохранять функциональный API и уменьшать стоимость массовых обновлений.

03

# Head retention

Почему lazy sequence может неожиданно удерживать память?

# Снова lazy-seq

```
(def xs
  (map (fn [x] (byte-array 10000))
       (range)))

(dorun (take 100000 xs)) ;; выделяется ~ 1ГБ памяти
```

# Идеальная потоковая обработка

`produce` → `consume` → `discard`

Память остаётся ограниченной, если обработанные элементы становятся unreachable.

# Head retention

`head → realized prefix → current position → …`

Ссылка на начало lazy sequence сохраняет достижимость уже обработанной части.

# Типичный паттерн ошибки

```
(let [xs (map f huge-input)]
  (reduce + xs)
  (first xs))
```

head stays reachable

Пока xs нужен после reduce, ссылка на head остается живой. GC не может освободить реализованную часть sequence.

# Head retention и время жизни локальных ссылок

## xs нужен после reduce

```
(let [xs (map f huge-input)]
  (reduce + xs)
  (first xs))
```

Начало seq удерживается
во время reduce.

## xs больше не нужен

```
(let [xs (map f huge-input)]
  (reduce + xs))
```

Ссылка на начало может
быть очищена до конца let.

Clojure: locals clearing — фактическое время жизни ссылки может быть короче области видимости.
GC сможет собирать префикс лишь при отсутствии других удерживающих ссылок.

# Как избегать head retention

| | | |
|---|---|---|
| не хранить head без нужды | ↔ | меньше живых ссылок |
| reduce / transduce | ↔ | потоковая агрегация |
| осторожно с def lazy seq | ↔ | долгоживущая root-ссылка |
| понимать realization | ↔ | предсказуемая память |

04

# Сборка мусора

Как runtime делает большое число короткоживущих объектов дешёвыми?

# Garbage Collection (GC)

**Сборка мусора — автоматическое управление памятью**

GC определяет объекты, которые больше недостижимы,
и может освобождать занимаемую ими память.

# GC roots — корни достижимости

**Корневые ссылки, с которых GC начинает обход объектов**

Примеры GC roots:

- Локальные ссылки в активных методах
- Статические поля классов
- Внутренние ссылки виртуальной машины
- Глобальные ссылки JNI (из нативного кода)

# GC roots: пример на Java

```
class Item { int value; }

class Demo {
  static Item shared = new Item();
  List<Item> items =
      new ArrayList<>(List.of(new Item(),
new Item()));

  static void run() {
    Demo local = new Demo();
    Item current = local.items.get(0);
    // здесь выполняется другой метод
    use(current);
  }
}
```

## GC roots

- static Demo.shared
- local и current в активном run()

# GC roots: пример на Clojure

```
(def shared {:id 1})

(defn process []
  (let [items [{:id 2} {:id 3}]
        current (first items)]
    ;; выполнение внутри process
    (println (:id current))
    (println (count items))))
```

## GC roots

```
GC root (стек JVM)
  items
  current

GC root (runtime JVM)
  namespace
  → Var #'shared
```

# Reachability

**GC roots → reachable objects**

Недостижимый объект = кандидат на освобождение памяти

# Tracing GC

```
roots → A → B → C
D → E
D → F
```

```
D, E, F не достижимы
из roots
 → могут быть собраны
```

# Ключевое различие

**`unreachable` ≠ «не нужен программе»**

GC освобождает только недостижимую память. Логически бесполезный, но достижимый объект остается.

# Generational GC

**Гипотеза поколений: большинство объектов живет недолго.**

`allocate` → `young generation` → `survive?` → `promote`

# Как используется гипотеза поколений?

**Новые объекты размещаются в молодом поколении (Young)**

**При сборке мусора: недостижимые объекты освобождаются, выжившие объекты сохраняются.**

**Объекты, пережившие несколько сборок, могут перейти в старшее поколение (Old)**

**Идея: часто собирать небольшую область с множеством короткоживущих объектов,**
**а долгоживущие проверять реже. Детали зависят от конкретного GC JVM.**

# Allocation на JVM — это дорого?

```
new Item()
```

**обычно нет**
**часто дешевле, чем думают**

# Что GC не решает

| | | |
|---|---|---|
| лишние аллокации | ↔ | нагрузка на GC |
| большой live set | ↔ | большое потребление памяти |
| долгие ссылки | ↔ | объекты остаются reachable |
| плохой алгоритм | ↔ | GC не спасает |

# Как всё связано

| **Что оптимизируем** | **Главный механизм** |
| --- | --- |
| Стоимость создания новой версии коллекции | Structural sharing |
| Количество достижимых, но ненужных объектов | Избегание head retention |
| Стоимость управления короткоживущими объектами | Generational GC |